A. Notelock

time limit per test 1 second
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 800 | Tags: greedy, two pointers | https://codeforces.com/problemset/problem/2154/A

Teto is playing the hit rhythm game *osu!*. The game can be described by a binary string[1] s of length n and a positive integer k where the following will happen in order:

- You will choose some positions in s to protect.
- Then for each i ($1 \le i \le n$) in increasing order, Teto can set s_i to mathtt0 if all the following are true:
- s_i = mathtt1,
- s_i is not protected,
- the previous k - 1 elements do not contain mathtt1. More formally, mathtt1 does not occur in $s_{\max(1,\, i - k + 1)}, \ldots, s_{i - 1}$.

You dislike Teto (for some reason). So determine the minimum number of positions you need to protect to force her to leave s unchanged.

[1]A binary string is a string that only consists of characters mathtt0 and mathtt1.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 100$). The description of the test cases follows.

The first line of each testcase contains integers n and k ($2 \le n \le 1000$; $2 \le k \le n$) — the length of s and k.

The second line of each test case contains a binary string s of length n consisting of characters mathtt0 and mathtt1.

The sum of n across all testcases does not exceed 1000.

Output

For each testcase, output the minimum number of positions you need to protect to force Teto to leave the string unchanged.

Example

Input

```
9
2 2
11
6 6
100001
5 3
10000
7 2
1010101
7 4
0000001
3 3
010
3 2
011
7 4
1001001
8 3
00000000
```

Output

```
1
1
1
4
1
1
1
1
0
```

Note

For the first testcase, you can protect the first element and have: s = mathttcolor*red*11. Now Teto cannot change s_1 because it is protected and cannot change s_2 because s_1 = mathtt1. It can be proven this is optimal.

For the second testcase, you can protect only the first element and have s = color*red*mathtt1mathtt00001. Teto cannot change s_1 because it is protected and she cannot change s_6 because there is mathtt1 in the previous k - 1 elements (color*blue*mathtt10000mathtt1).

For the fourth testcase, you must protect s_1,s_3,s_5,s_7 and have s = mathttcolor*red*10color*red*10color*red*10color*red*1. It can be shown that this is optimal. For example, if you did not protect s_3, then Teto can change it to mathtt0 (mathttcolor*red*1color*blue*010color*red*10color*red*1)